# Semana 04 — Escaner propio frente a nmap

Grupo 4 · rol Rojo · ejemplo de trabajo

Prof. César Rodríguez | ELECTRONICA | 1 de noviembre de 2026

| | |
|---|---|
| **Curso** | CYBERSEGURIDAD — Ciberseguridad Aplicada |
| **Semana** | 04 de 10 |
| **Fase** | 2 — Ofensiva controlada |
| **Grupo** | 4 |
| **Rol** | Rojo |
| **Entrega** | GitHub, carpeta semana_04/ |
| **Corte** | Domingo 23:59 de la semana correspondiente; lo posterior no se corrige. |

## 1 Tu rol en la práctica

Ejecuta la técnica ofensiva sobre el objetivo autorizado: prepara el entorno, lanza el ataque, registra comandos y payloads, y guarda cada salida con su hash.

- Confirmar el objetivo autorizado y preparar el entorno antes de la primera captura.
- Ejecutar la técnica de la semana en el orden de los pasos y guardar cada salida cruda con su hash.
- Redactar la reproducción: comando, salida esperada y salida obtenida, sin retocar la evidencia.

## 2 Qué aporta este rol al entregable

Evidencia cruda en semana_NN/evidencia y el apartado de reproducción del informe.

## 3 Ejemplo paso a paso

El rol Rojo construye el escaner, lo ejecuta contra el laboratorio y lo compara con nmap. La comparacion tiene que explicar las diferencias, no solo listarlas.

### 3.1 Implementar sondeo TCP connect con tiempo de espera configurable

El script recibe host, puertos y timeout, y devuelve JSON normalizado. Un puerto cerrado y un timeout son resultados distintos y se reportan por separado.

```
mkdir -p semana_04/evidencia
cat > semana_04/escaner.py <<'PY'
import argparse, json, socket, time

def connect(host, puerto, timeout):
    inicio = time.perf_counter()
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(timeout)
    try:
        sock.connect((host, puerto))
        estado = 'abierto'
    except socket.timeout:
        estado = 'timeout'
    except ConnectionRefusedError:
        estado = 'cerrado'
    finally:
        sock.close()
    return {'puerto': puerto, 'estado': estado,
            'ms': round((time.perf_counter() - inicio) * 1000, 1)}

ap = argparse.ArgumentParser()
ap.add_argument('--host', default='127.0.0.1')
ap.add_argument('--puertos', default='8081,3000,2222')
ap.add_argument('--timeout', type=float, default=0.5)
args = ap.parse_args()
resultados = [connect(args.host, int(p), args.timeout)
              for p in args.puertos.split(',')]
print(json.dumps(resultados, indent=2))
PY
python3 semana_04/escaner.py > semana_04/evidencia/propio.json
```

## 3.2 Añadir sondeo UDP y documentar el SYN

UDP se prueba enviando un datagrama y esperando ICMP. El SYN semiabierto exige raw sockets y privilegios: si el entorno no lo permite, se documenta la limitacion y se compara con nmap -sS en lugar de simularlo.

```
python3 -c "
import socket
s=socket.socket(socket.AF_INET,socket.SOCK_DGRAM); s.settimeout(0.5)
s.sendto(b'ping',('127.0.0.1',3000))
try: print('respuesta', s.recvfrom(64))
```

```
except socket.timeout: print('sin respuesta en 500 ms')
" | tee semana_04/evidencia/udp.txt
sudo nmap -sS -p 8081,3000,2222 127.0.0.1 -oN semana_04/evidencia/
    syn_nmap.txt
```

### 3.3 Comparar y medir tiempos

Se ejecuta nmap sobre el mismo objetivo y se tabulan coincidencias, diferencias y tiempo por tecnica. Un falso positivo es un puerto que el script da por abierto y nmap no confirma.

### 3.4 Explicar los falsos positivos

Cada falso positivo tiene una causa concreta: timeout demasiado corto, servicio que acepta y cierra, o filtrado que responde distinto. La explicacion se escribe junto al resultado.

### 3.5 Tecnica, resultado propio y contraste con nmap

**Tecnica, resultado propio y contraste con nmap**

| Tecnica | Propio | nmap | Tiempo propio | Falso positivo |
|---|---|---|---|---|
| TCP connect | Abierto 8081, 3000, 2222 | Abierto en los tres | Menos de 1 s | Ninguno observado |
| TCP SYN | No implementado sin privilegios | Abierto en los tres | Menos de 1 s | Se documenta la limitacion |
| UDP | Sin respuesta en 500 ms | Abierto o filtrado | 500 ms por puerto | No se puede distinguir con un paquete |
| Timeout corto | Cerrado por timeout | Abierto | 50 ms | Si: el limite era menor que la latencia |

Los tiempos son de esta maquina y esta sesion; se reportan para comparar tecnicas entre si, no como valor absoluto.

## 4 Qué guardar en el repositorio

Se entrega el script con type hints y docstrings, la salida JSON, la comparacion con nmap y la explicacion de cada diferencia.

```
semana_04/
|-- informe.md
|-- escaner.py
|-- evidencia/
```

```
|   |-- propio.json
|   |-- syn_nmap.txt
|   |-- udp.txt
|   `-- comparativa.txt
`-- README.md
```

- El script acepta host, puertos y timeout por linea de comandos.
- Un puerto cerrado y un timeout se reportan como estados distintos.
- La comparacion incluye tiempo por tecnica y explica las diferencias.
- Las limitaciones de privilegios estan declaradas, no simuladas.

## 5 Cómo se corrige

| Criterio | Peso |
|---|---|
| Huella digital completa (DNS/WHOIS) | 2,0 |
| Escáner funcional con técnicas solicitadas | 3,0 |
| Comparación rigurosa contra nmap | 2,5 |
| Explicación técnica de falsos positivos y sigilo | 1,5 |
| Código con type hints, docstrings y manejo de errores | 1,0 |
| **Total** | **10,0** |

- El escaneo se ejecuta solo contra 127.0.0.1 y los puertos del alcance.
- {'Un falso positivo no se corrige bajando el timeout a ojo': 'se mide y se justifica.'}

**Recuerda**

El trabajo es independiente: cada grupo usa su propio repositorio y su propio laboratorio, y la comparación de resultados es en clase. Corte: Domingo 23:59 de la semana correspondiente; lo posterior no se corrige.